



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



EvilPayout Wiper Sample Analysis

Malware Analysis

Classification	TLP:CLEAR
Published	2026-06-18
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis

TLP:CLEAR | Lost Boys Cyber | EvilPayout Wiper Sample Analysis - Malware Analysis

Published: 2026-06-18 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Sample Metadata
3. Source Review & Confidence
4. Initial Triage and Delivery Assessment
5. Static Analysis
 - 5.1. File and Build Characteristics
 - 5.2. Strings, Imports, and Embedded Artefacts
 - 5.3. Static Configuration Clues
6. Code Analysis
 - 6.1. Reverse-Engineering Method
 - 6.2. Functions and Logic of Interest
 - 6.3. Anti-Analysis, Evasion, and Obfuscation
7. Dynamic Analysis
 - 7.1. Execution Overview
 - 7.2. Filesystem, Registry, Service, and Task Changes
 - 7.3. Network and Command-and-Control Behaviour
 - 7.4. Observed Capability Summary
8. Process Tree and Execution Chain
9. Persistence and System Modification
10. Network and Infrastructure Analysis
11. Indicators of Compromise
12. MITRE ATT&CK Mapping
13. Detection Engineering
 - 13.1. Detection Logic Summary
 - 13.2. Detection Content
 - 13.3. Detection Validation and Blind Spots
14. Threat Hunting
 - 14.1. Hunt Hypotheses
 - 14.2. Hunt Queries
 - 14.3. Triage Guidance for Hunt Hits
15. Triage and Incident Response
16. Validation and Emulation
17. Recommendations
18. References and Refresh Notes

EvilPayout Wiper Sample Analysis

1. Executive Summary

The submitted sample `42ed646eed4f...` is best assessed as a destructive Windows wiper aligned with the EvilPayout intrusion set. This is the strongest of the three attributions because the local evidence is unusually specific: the binary is a PE32 .NET console executable with an embedded PDB path of `C:\work\wiper\Wiper\obj\Release\Wiper.pdb`, and static triage recovered the strings `meciwipe.ini`, `preg.exe`, `wevtutil.exe`, and `DestroyMBR`. Those artefacts map closely to public reporting on EvilPayout destructive tooling.

The sandbox verdict for this sample was `malicious` with the highest evidence score of the three submissions. Runtime evidence was still imperfect and included a likely noisy `OneDrive` autorun, but the static and external corroboration is sufficient to treat this as destructive malware with potential disk- or boot-impact capability. The presence of `wevtutil.exe` also supports log-clearing behavior that would reduce defender visibility during or after impact.

Directly observed evidence drives this report: hashes, PE/.NET identity, PDB path, and destructive artefact strings. High-confidence assessment: the sample is wiper malware associated with EvilPayout reporting. Lower-confidence point: the exact persistence sequence seen in the sandbox may not belong to the sample and should be revalidated on any real host before being used in an incident timeline.

2. Sample Metadata

Field	Value
Family	EvilPayout
Malware Type	Wiper
Platform	Windows x86 / .NET
Primary Sample SHA256	`42ed646eed4f949c456c637a222e7d94dd8ac67ed5ebda5e63c7b7979076d9cf`
Additional Hashes	SHA1 `245961ca2c0646b2e0926e5281bc3398aceac7be`; MD5 `cb14d849149831145963a44950a12851`
File Type / Format	PE32 executable (console), Intel i386 Mono/.NET assembly
Delivery Theme	Unknown from the recovered archive alone
First Seen / Reported	2026-06-17 internal submission
Analyst Confidence	High
Primary Objective	Destructive impact, log reduction, and potential boot/disk tampering

3. Source Review & Confidence

Source	Contribution	Confidence
Internal sandbox analysis	Verdict, telemetry context, ATT&CK rollup	High
Internal static triage	PDB path, artefact strings, hashes, PE characteristics	High
External EvilPayout reporting	Exact-hash and family-context	Medium

	corroboration	
--	---------------	--

Observed evidence alone is enough to classify the sample as destructive malware. External reporting mainly strengthens the campaign label and provides historical context.

4. Initial Triage and Delivery Assessment

The file was recovered from a password-protected analyst-handled archive. No companion installer or lure artefacts were provided. The payload itself, however, contains enough uniquely identifying material to anchor the family call without relying on a broader intrusion chain.

Variant / Artefact	Delivery Role	Notes
`42ed646eed4f...sample`	Primary payload	.NET PE32 executable used for local static and sandbox review.
Password-protected `.7z` archive	Transfer wrapper	Safe submission format, not proof of initial delivery method.

5. Static Analysis

5.1. File and Build Characteristics

The sample is a 766,464-byte PE32 .NET console executable with three sections (`.text`, `.rsrc`, `.reloc`). `rabin2` reported `lang cil` and preserved the debug path `C:\work\wiper\Wiper\obj\Release\Wiper.pdb`, which is highly probative. The compile timestamp is not trusted as an attribution anchor, but it does not reduce confidence in the destructive role.

Static Signal	Evidence	Analyst Value
Family-specific debug path	`C:\work\wiper\Wiper\obj\Release\Wiper.pdb`	Strong developer/build clue directly supporting the wiper assessment.
Destructive routine marker	`DestroyMBR` string	Indicates potential overwrite or tampering of the master boot record or equivalent boot structures.
Support artefacts	`meciwipe.ini`, `pregg.exe`, `wevtutil.exe` strings	Links the sample to configuration, helper tooling, and log-clearing behavior.

5.2. Strings, Imports, and Embedded Artefacts

The import table is minimal because the file is a .NET assembly and primarily surfaces through `\_CorExeMain`. That makes the recovered strings and PDB path especially valuable. The combination of `DestroyMBR`, configuration/helper artefacts, and the wiper project path is more operationally meaningful than the sparse native import table.

5.3. Static Configuration Clues

`meciwipe.ini` likely represents configuration or operational guidance for the destructive workflow. `pregg.exe` appears to be an additional binary or helper artefact. `wevtutil.exe` strongly suggests log clearing or pre-impact cleanup. These strings were preserved in the curated IOC set because they are materially useful for hunting.

6. Code Analysis

6.1. Reverse-Engineering Method

Validation relied on `file`, cryptographic hashes, `rabin2`, curated string review, and the internal `static\_triage.json` output. Ghidra MCP was unavailable in this session, so no decompiler-backed .NET routine naming was produced. Public Check Point reporting was used to bridge the family link because it referenced the same hash and matching artefacts.

6.2. Functions and Logic of Interest

Function / Routine	Evidence	Why It Matters
.NET entrypoint	`_CorExeMain` import	Confirms managed .NET execution rather than a pure native PE.
MBR-impact routine	`DestroyMBR` string	Strong clue that the malware can perform destructive disk or boot operations.
Event-log clearing	`wevtutil.exe` string	Suggests anti-forensics or operational cleanup before/after destructive action.

6.3. Anti-Analysis, Evasion, and Obfuscation

No advanced anti-analysis chain was proven from the available local evidence, but the minimal native import table and reliance on .NET internals limit what can be concluded without deeper decompilation.

7. Dynamic Analysis

7.1. Execution Overview

The sandbox run rated the sample as malicious and recorded the richest ATT&CK coverage of the three. Runtime telemetry still lacked a full process tree or clean destructive event replay, so the report remains anchored to the stronger static evidence.

7.2. Filesystem, Registry, Service, and Task Changes

No validated dropped files were recovered, but the sandbox summary recorded a suspicious `OneDrive` autorun under a user-writable path. Because similar noise appeared in other runs, that persistence observation is retained only as a secondary lead rather than a primary conclusion.

7.3. Network and Command-and-Control Behaviour

As with the other runs, observed DNS and TLS traffic was dominated by background Windows communications. No stable malicious C2 was confirmed. This is not unusual for a destructive payload whose main objective is impact rather than sustained beaoning.

7.4. Observed Capability Summary

Capability	Supporting Dynamic Evidence
Potential persistence or staging	Sandbox summary flagged a user-writable `Run` key path, though attribution to the sample is cautious.
Destructive execution lead	Overall malicious verdict plus strongly aligned static strings point toward impact-oriented behavior even without full runtime replay.

8. Process Tree and Execution Chain

No trustworthy guest process tree was available, so the final chain remains intentionally conservative.

Step	Parent Process	Child / Action	Why It Matters
1	Unverified	Direct execution of submitted .NET payload	Confirms only lab execution.
2	Unverified	Possible follow-on log clearing or destructive staging	Supported by recovered strings, not by a full captured process tree.

9. Persistence and System Modification

The sample's primary risk is destructive impact, not stealthy long-term persistence. The only candidate persistence artefact in local telemetry is the user-writable `OneDrive` autorun, which should be hunted but not over-claimed.

Modification Type	Artefact	Evidence	Confidence
Registry Run key	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\OneDrive`	Sandbox summary recorded a user-writable autorun target	Medium
Anti-forensics / cleanup	`wevtutil.exe`	Static artefact indicates likely event-log clearing support	High

10. Network and Infrastructure Analysis

No stable C2 or exfiltration infrastructure was validated. The meaningful operational artefacts are host-based rather than network-based.

IOC / Infrastructure	Type	Role	Stability / Confidence
`C:\work\wiper\Wiper\obj\Release\Wiper.pdb`	Filepath	Build-path clue strongly tied to the sample family	High
`meciwipe.ini`	Filename	Destructive tooling/config clue	High
`preg.exe`	Filename	Associated helper artefact	High
`DestroyMBR`	String	Destructive capability marker	High

11. Indicators of Compromise

The curated indicator set emphasizes sample hashes and high-value host artefacts rather than noisy auto-extracted pseudo-domains. This makes the report more actionable for EDR and DFIR teams.

IOC Type	Example / Count	Source	Operational Notes
SHA256	1	Observed	Primary exact-match sample key.
SHA1 / MD5	2	Observed	Useful for repository and legacy-tool correlation.
File paths / names / strings	5	Observed	Best host-based hunting material for this family.

12. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1561.001	Disk Structure Wipe	`DestroyMBR` supports destructive disk or boot-structure impact.
T1070.001	Clear Windows Event Logs	`wevtutil.exe` indicates likely log-clearing behavior.
T1060	Registry Run Keys / Startup Folder	Candidate `OneDrive` autorun appeared in

		the sandbox summary.
T1059.001	PowerShell	Sandbox ATT&CK rollup flagged PowerShell-like content, though this was not the strongest local finding.

13. Detection Engineering

13.1. Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes
Wiper-family strings and PDB path	File/YARA scanning	Catch the exact sample or close variants on disk	High-confidence static detection path for this sample family.
Destructive-tool command lines	Process telemetry	Surface pre-impact log clearing or helper execution	Tune against administrative maintenance tools.

13.2. Detection Content

Primary detection content is stored in ``detections/evilplayout-wiper.yara`` and ``detections/evilplayout-wiper-sigma.yml``.

13.3. Detection Validation and Blind Spots

The YARA rule should be effective for the analyzed sample and variants retaining the same build-path and helper strings. The Sigma rule may fire on legitimate administrator use of ``wevtutil``; tune using parent process lineage, host role, and co-occurring strings.

14. Threat Hunting

14.1. Hunt Hypotheses

- Hosts affected by this malware family will expose ``meciwipe.ini``, ``preg.exe``, ``DestroyMBR``, or ``wevtutil`` command-line evidence before or during destructive activity.
- Even if disk impact does not trigger in the sandbox, enterprise telemetry may retain traces of staging, helper binaries, or masquerading autoruns.

14.2. Hunt Queries

See ``detections/hunts.kql`` and ``detections/hunts.spl`` for the prepared hunt content.

14.3. Triage Guidance for Hunt Hits

Any host with these family-specific strings or helper artefacts should be escalated immediately. Prioritize forensic preservation and boot/disk integrity checks before rebooting or running automated cleanup, because destructive malware can render post-incident collection much harder.

15. Triage and Incident Response

Immediately isolate confirmed hosts, preserve disk and volatile evidence, and validate whether event logs were cleared or storage structures altered. Because EvilPloy-style activity is destructive and potentially coordinated, scope laterally for related staging hosts, operator tooling, and identity abuse before restoring affected systems.

16. Validation and Emulation

Validation for this report includes sandbox review, local static triage, and exact-hash corroboration from public EvilPloy reporting. Deeper .NET decompilation was not completed in this session due the Ghidra MCP blocker, so the report should be refreshed if full code-level recovery becomes available.

17. Recommendations

Priority	Owner	Action	Rationale
Critical	IR / SOC	Isolate any host matching the sample or artefacts and preserve disk images before remediation	This sample is assessed as destructive wiper malware.
Critical	Detection Engineering	Deploy YARA and process-creation detections for `DestroyMBR`, `meciwipe.ini`, `preg.exe`, and `wevtutil` patterns	These are the strongest family-specific artefacts recovered locally.
High	Infrastructure / IT	Validate backup integrity and recovery procedures for systems in the same trust boundary	Wiper operations prioritize impact over persistence.

18. References and Refresh Notes

- Internal sandbox and static triage artefacts listed in `sources.md`
- Check Point and MalwareBazaar exact-hash references
- Refresh if full .NET decompilation or victim-side destructive telemetry becomes available